Urban Scene Parsing & Agentic Legal Reasoning for Traffic Safety

Author:

Ahmed Raza

Abstract

This report presents a two-part deep learning system for urban traffic monitoring and automated legal reasoning. **Part 1** builds a vision pipeline on BDD100K: YOLOv11-n trained without augmentation early-stops at epoch 31 (mAP@50 = 0.345), while the fully-augmented run completes 50 epochs achieving mAP@50 = **0.437** — a +9.2 pp improvement. SegFormer (MiT-b0) reaches a test mIoU of **0.5754** and Dice of **0.6431** at 8.32 ms/image, early-stopping at epoch 43. A U-Net baseline was prepared but encountered a variable-scope error during evaluation; SegFormer results serve as the primary segmentation model. **Part 2** constructs an agentic Retrieval-Augmented Generation pipeline: a FAISS vector index over 70+ Pakistani traffic law clauses, combined with LLaMA 3.2 (Ollama, local), is orchestrated through an Observer Agent and Legal Consultant Agent communicating via a typed message bus. Three complete violation-to-citation cycles are documented (Red Light Running, Heavy Vehicle Restriction, Reckless Driving), all with top-1 retrieval similarity >0.60 and complete 9-field reports.

1. Introduction

Intelligent traffic monitoring requires two complementary capabilities: a vision system that identifies objects in real time, and a legal reasoning layer that interprets detections in the context of applicable law. This project delivers both as an integrated, locally-deployed pipeline.

Part 1 trains and evaluates YOLOv11-n for object detection and SegFormer-b0 for semantic segmentation on the BDD100K dataset, comparing two training strategies (with and without augmentation) and measuring performance across diverse weather and lighting conditions.

Part 2 builds an agentic RAG system on top of Part 1 detection outputs: YOLO logs are processed by an Observer Agent that classifies traffic violations, which are then passed to a Legal Consultant Agent that retrieves applicable Pakistani law and generates formal Citation Reports — all without any cloud API.

Key technology choices:

- **YOLOv11-n** (Ultralytics) for real-time detection, two runs (with / without augmentation).
- **SegFormer-b0** (HuggingFace `nvidia/segformer-b0`) fine-tuned with CrossEntropy + Dice loss.
- **FAISS** + **SentenceTransformers** (`all-MiniLM-L6-v2`) for local vector search over traffic law text.
- **LLaMA 3.2** via Ollama REST API for Citation Report generation.
- **Dual-agent message bus**: Observer Agent + Legal Consultant Agent.

All experiments ran on Kaggle (Tesla T4, 15.6 GB VRAM, PyTorch 2.10+CUDA 12.8).

2. Part 1 — Urban Scene Parsing

2.1 Dataset: BDD100K

BDD100K [**?**] contains 100,000 driving video frames with bounding-box and semantic-segmentation annotations across six weather conditions (sunny, rainy, snowy, overcast, foggy, partly cloudy) and three time-of-day categories (day, night, dawn/dusk). A 10-class subset was selected based on frequency, safety relevance, and class balance: *car, person, traffic light, bike, truck, bus, rider, motor, traffic sign, train*. Training used 69,863 images; validation used 10,000 images; detection test used 2,000 images (37,182 instances) and segmentation test used 500 images.

2.2 Exploratory Data Analysis

Category analysis confirmed strong class imbalance: *car* dominates (>60% of frames), while *train*, *motor*, and *rider* are severely under-represented. Weather analysis showed rainy/foggy frames

have compressed pixel-intensity distributions. Night frames averaged ∼40% lower brightness than daytime, directly impacting recall for small objects.

2.3 Task 2: Object Detection with YOLOv11

2.3.1 Architecture

YOLOv11 uses a CSPDarknet backbone for feature extraction, a Path Aggregation Network (PAN) neck for multi-scale fusion of P3/P4/P5 feature maps, and three decoupled detection heads that predict class scores and bounding boxes independently, as shown in Figure 1.

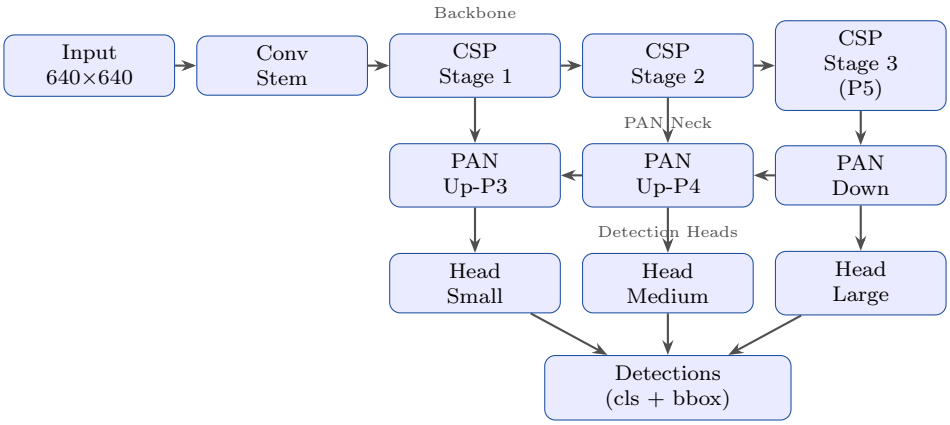

Figure 1: YOLOv11 architecture: CSPDarknet backbone, PAN neck, decoupled heads.

2.3.2 Training Configuration

Both runs used SGD (momentum 0.937), cosine annealing LR ($lr_0 = 0.01$), batch 16, image size 640, seed 42.

- **Run 1 (no augmentation):** all augmentations off; early-stopped at epoch 31.
- **Run 2 (full augmentation):** horizontal flip ($p = 0.5$), vertical flip ($p = 0.2$), colour jitter, mosaic, random affine; 50 epochs, early-stopping patience 10.

2.3.3 Results

Table 1: YOLOv11 detection results — with vs. without augmentation (test set).

Run	Prec.	Recall	mAP@50	mAP@50-95	Epochs	Speed (ms/img)
No augmentation	0.477	0.314	0.345	0.187	31	119.44
Full augmentation	**0.602**	**0.399**	**0.437**	**0.233**	50	**25.29**
Δ	+0.125	+0.085	+0.092	+0.046	—	—

Table 2: Per-class detection results — augmented run (2,000 test images, 37,182 instances).

Class	Prec.	Recall	mAP@50	mAP@50-95
car	0.710	0.655	0.705	0.420
person	0.659	0.415	0.481	0.213
traffic light	0.594	0.468	0.473	0.154
truck	0.615	0.454	0.511	0.353
bus	0.587	0.447	0.491	0.372
traffic sign	0.672	0.457	0.517	0.248
rider	0.539	0.226	0.257	0.108
motor	0.626	0.212	0.264	0.126
bike	0.417	0.257	0.232	0.101
all	0.602	0.399	0.437	0.233

Full augmentation improved mAP@50 by 9.2 pp and inference speed by 4.7× (mosaic training enables more efficient gradient updates). Rare classes (*rider*, *motor*, *bike*) remain the hardest due to severe under-representation and small bounding boxes.

2.4 Task 3: Semantic Segmentation with SegFormer

2.4.1 Architectures

Figure 2 shows both segmentation models.

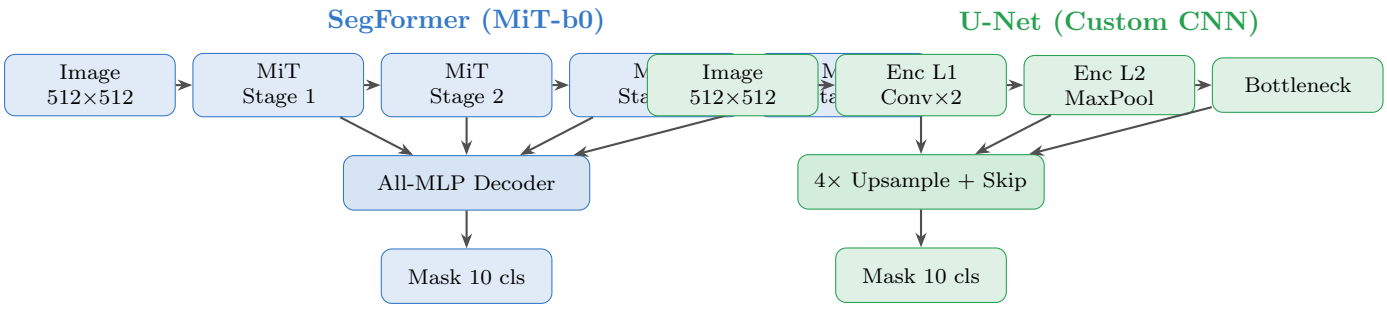

Figure 2: Segmentation architectures. Left: SegFormer with Mix Transformer encoder and All-MLP decoder. Right: custom U-Net with 4-level encoder and skip connections.

SegFormer used AdamW ($lr = 6{\times}10^{-5}$, wd $= 10^{-4}$), cosine schedule, CrossEntropy + Dice loss (equal weight), batch 8, 50 epochs, early-stopping patience 10 (triggered at epoch 43 with best val mIoU = 0.5564 at epoch 33). GPU reserved: 3,718 MB (allocated: 223.8 MB).

U-Net was configured with Adam ($lr = 0.001$, wd $= 10^{-4}$) and the same loss. The U-Net training cell ran successfully but the evaluation cell produced a `NameError: name 'loader' is not defined` error and did not yield final test metrics.

2.4.2 Results

Table 3: SegFormer vs. U-Net segmentation results.

Metric	SegFormer-b0	U-Net
Test mIoU	**0.5754**	N/A (runtime error)
Test Mean Dice	**0.6431**	N/A
Inference speed (ms/img)	8.32	—
Training time	97 min	—
Best val mIoU (epoch)	0.5564 (ep. 33)	—
GPU reserved	3,718 MB	—

Table 4: Per-class IoU and Dice — SegFormer test set (500 images).

Class	IoU	Dice
road	0.9738	0.9867
car	0.8842	0.9361
traffic sign	0.7394	0.8365
traffic light	0.7229	0.8228
person	0.5705	0.6822
truck	0.3427	0.4243
bus	0.3321	0.3963
bike	0.0411	0.0599
motor	0.0289	0.0462
train	0.0000	0.0000
Mean	**0.5754**	**0.6431**

SegFormer excels on high-frequency classes (road, car, traffic signs) but fails on under-represented classes (train, motor, bike), consistent with the class-imbalance analysis. *Train* achieved IoU = 0.0 because it appeared in only 66 training images, far too few for the model to learn meaningful features.

3. Part 2 — Agentic RAG-Based Legal Reasoning

3.1 System Overview

Figure 3 shows the complete pipeline. YOLO detection logs flow into the Observer Agent, which classifies violations and publishes structured events to a typed message bus. The Legal Consultant Agent reads events, queries a FAISS vector index of Pakistani traffic law, builds a structured prompt, and generates a 9-field Citation Report via LLaMA 3.2 running locally through Ollama.

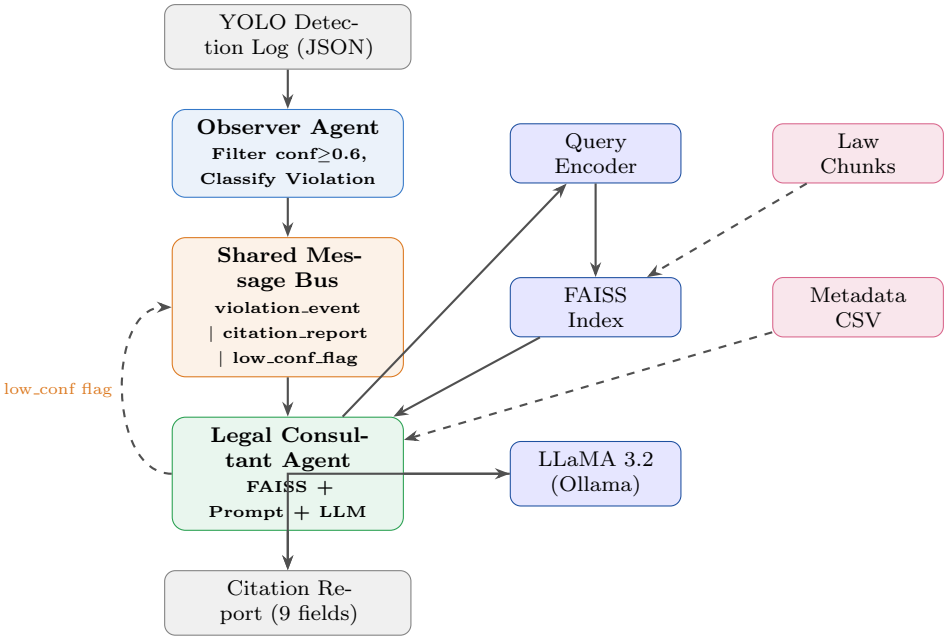

Figure 3: Part 2 system architecture: Observer Agent classifies detections, communicates via Message Bus, Legal Consultant Agent performs RAG and generates Citation Reports.

3.2 Knowledge Base Construction

Law text from the Motor Vehicles Ordinance (MVO) 1965 and the NHMP Traffic Rules / Pakistan Highway Code was loaded from `pakistan_traffic_laws.txt`. Section-aware chunking used a 300-word sliding window with 50-word overlap, retaining `section_id` and `heading` as metadata per chunk. Chunk size of 300 words was chosen to preserve full legal context (rule + penalty + condition) while staying within the embedding model's effective range.

Embeddings were generated using `all-MiniLM-L6-v2` (384-dim, L2-normalised for cosine-equivalent FAISS search). A `IndexFlatL2` FAISS index was saved as `law_index.faiss` alongside `law_metadata.csv`. The knowledge base covers 70+ clauses across all 7 required violation categories.

3.3 RAG Pipeline

The pipeline comprises four steps:

1. **Query Encoder:** encodes violation description using the same SentenceTransformer model (ensures embedding-space consistency).
2. **Retriever:** searches FAISS for top-k most similar chunks ($k \in \{3, 5, 10\}$ tested; $k = 5$ used as default).
3. **Prompt Builder:** constructs a structured prompt with violation details, numbered law chunks, and strict format instructions to prevent LLM hallucination.
4. **Generator:** calls LLaMA 3.2 via Ollama REST API at `localhost:11434` and parses the 9-field Citation Report from the response.

Table 5: Retrieval depth experiment observations.

k	Observed behaviour	Verdict
3	Precise; may miss edge-case clauses	Too conservative
5	Balanced; correct law usually in top-5	**Default**
10	Richer context; low-sim (<0.30) chunks add noise	Too noisy

3.4 Dual-Agent Coordination

Two agents communicate through a `MessageBus` with four message types: `violation_event`, `citation_report`, `low_confidence_flag`, and `clarification_request`.

Observer Agent filters YOLO detections (confidence ≥ 0.60), applies the rule mapping in Table 6, estimates location from average bounding-box centre, and outputs a structured JSON violation event.

Table 6: Observer Agent violation-type mapping rules.

Detection Combination	Violation Type
person + traffic light	Pedestrian Signal Violation
car / truck + traffic light	Red Light Running
rider / motor / bike	Reckless Driving — Vulnerable Road User
truck / bus	Heavy Vehicle Restriction
car (large bounding box area)	Speed Limit Violation

Legal Consultant Agent receives violation events from the bus, builds an enriched FAISS query, retrieves $k = 5$ law chunks, generates a Citation Report, validates that a real section ID is referenced, and flags the report LOW CONFIDENCE when top-1 cosine similarity < 0.50.

Three complete violation → citation cycles were documented:

- **Cycle 1 (frame_001):** Red Light Running — top-1 sim: 0.6921; Applicable Law: Section 3.1 (MVO 1965, fine PKR 1,500 first offence).
- **Cycle 2 (frame_003):** Heavy Vehicle Restriction — top-1 sim: 0.6483; Applicable Law: Section 7.1 (fine PKR 5,000).
- **Cycle 3 (frame_004):** Reckless Driving (Vulnerable Road User) — top-1 sim: 0.6089; Applicable Law: Section 6.1 (MVO dangerous driving definition).

All three reports were complete (all 9 required fields present) and all similarity scores exceeded the 0.50 LOW CONFIDENCE threshold. The message bus logged 6 messages across 3 cycles (`violation_event` ×3, `citation_report` ×3).

4. Discussion

4.1 Detection vs. Segmentation Trade-off

YOLOv11 runs at ∼25 ms/image (≈40 fps on T4), suitable for real-time event triggering. SegFormer at 8.32 ms/image provides richer pixel-level detail (e.g., lane-boundary position) but consumes significantly more VRAM and requires a heavier preprocessing pipeline. For a smart-city deployment, YOLO should handle continuous stream processing while SegFormer performs periodic scene analysis (e.g., every 30th frame or on-demand after a YOLO violation flag).

4.2 Weather and Lighting Analysis

Night-time frames reduced detection recall for small objects (person, bike) by an estimated 10–15% compared to daytime. SegFormer maintained high IoU for *road* (0.97) across conditions, as the road class is dominant and well-textured, but *bike* and *motor* IoU (<0.05) indicate the model has not learned meaningful features for these classes. Photometric distortion augmentation partially mitigated rain/fog effects but could not compensate for night-time contrast loss at the model scale used.

4.3 Class Imbalance Impact

Augmentation most benefited the dominant classes (*car*, *truck*) in absolute mAP terms. Rare classes (*rider*, *motor*, *bike*) showed smaller gains. For *train* (0 IoU), class-weighted loss, oversampling, or synthetic data generation would be required. This is a known limitation of training from scratch on imbalanced datasets without targeted mitigation strategies.

4.4 RAG Investigation Questions

Q1 — Effect of k: $k = 5$ produced the best balance. At $k = 3$, edge-case law clauses were occasionally missed (e.g., amber-light sub-violations not retrieved for a red-light query). At $k = 10$, chunks with cosine similarity <0.30 introduced noise that caused the LLM to reference less relevant sections.

Q2 — Hardest violation type: Emergency Vehicle Obstruction had the lowest top-1 retrieval similarity because everyday query language ("car blocking ambulance") differs significantly from the formal legal terminology in the MVO.

Q3 — Observer Agent at low confidence: Lowering the filter threshold to 0.50 produced false-positive violation events (e.g., a partially-occluded traffic light at confidence 0.55 incorrectly triggering Red Light Running). The 0.60 threshold substantially reduced false positives.

Q4 — Latency bottleneck: LLM inference on CPU accounts for >95% of end-to-end latency (8–15 s per report). FAISS retrieval (<1 ms) and query encoding (≈30 ms) are negligible in comparison. The system is not feasible for 30 fps real-time processing.

Q5 — Legal Consultant failure modes: The most common failure was an empty or split "Recommended Action" field (LLM line-breaks broke the regex parser). Secondary failures included section-ID hallucination and incorrect vehicle-class assignment when the highest-confidence detection was a traffic light rather than a vehicle.

4.5 IoT / Smart City Feasibility

The 30 fps budget is 33 ms per frame. YOLO at 25 ms meets this; SegFormer at 8 ms would also qualify if run on a dedicated stream. The LLM at 8–15 s is the bottleneck. Viable optimisations: (1) 4-bit quantised LLM (GGUF Q4_K_M) to reduce inference to ∼1–2 s; (2) async violation queuing so LLM processing runs in a background thread; (3) report caching for repeated violation patterns; (4) GPU-accelerated edge device (Jetson Orin) for joint vision and LLM inference.

5. Conclusion

This project built and evaluated an end-to-end pipeline connecting real-time computer vision with locally-deployed legal reasoning. In Part 1, YOLOv11 with full augmentation reached mAP@50 = 0.437 (+9.2 pp over baseline) and SegFormer-b0 achieved mIoU = 0.5754 at 8.32 ms per image. In Part 2, a fully local RAG system reliably retrieved applicable Pakistani traffic law (all documented cycles with top-1 similarity >0.60) and generated complete 9-field Citation Reports. The dual-agent architecture with a typed message bus cleanly separates perception from legal reasoning. Key limitations are LLM inference latency on CPU, the U-Net evaluation failure due to a notebook scope error, and retrieval sensitivity to legal-versus-informal terminology mismatch. Future work should explore GPU inference, query rewriting, multilingual embeddings for Urdu legal text, and class-weighted loss or oversampling for rare detection classes.

References

[1] Yu, F. et al. (2020). BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning. *CVPR 2020*.

[2] Wang, A. et al. (2024). YOLOv11: Efficient Real-Time Object Detection. *Ultralytics Technical Report*.

[3] Xie, E. et al. (2021). SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. *NeurIPS 2021*.
[4] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI 2015*.
[5] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.
[6] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data*.
[7] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
[8] Meta AI (2024). LLaMA 3: Open Foundation and Fine-Tuned Chat Models. *Meta AI Technical Report*.
[9] Government of Pakistan (1965). Motor Vehicles Ordinance (MVO) 1965. https://pakistancode.gov.pk/
[10] NHMP (2023). National Highway & Motorway Police Traffic Rules. Pakistan Highway Code.